Human-Centered Neuroengineering LAB

C Programming (1)

2nd week. Recall last week

Department of Embedded System Engineering
Incheon National University

Instructor: Kyungho Won
kyungho.won@inu.ac.kr
Human-Centered Neuroengineering (HCN) Lab

INU 인천대학교 INCHEON NATIONAL UNIVERSITY

* These lecture slides are for class use only. Unauthorized distribution or modification is prohibited.

Recall last week: Practice Problems

Practice 1: Print Your Info

- Write a program that prints:
- Your name
- Your student ID number
- Your department

- Requirements:
- Use a separate printf() for each line
- Each line must end with ₩n

Practice 2: Simple Arithmetic

- Declare two integer variables a = 15, b = 27
- Calculate and print:

```
Sum:        a + b

Difference: a - b

Product:    a * b
```

- Each result on its own line

Practice 3: Float Formatting

- Declare: float pi = 3.141593;

- Print pi three times with different precision:

```
Default (6 decimal places): 3.141593

2 decimal places:           3.14

4 decimal places:           3.1416
```

- Hint: use %f, %.2f, %.4f

Practice 4: Draw a Pattern

- Print the following shape using printf():

```
*****

*  *

*  *

*****
```

- Use 4 printf() statements — one per line
- Pay attention to spaces inside the quotes!

Practice 5: Fix the Bug

- The following code has 3 errors. Find and fix them:

```
#include <stdio.h>

int main(void)
{
    Printf("Hello World\n")
    return 0
}
```

Practice 6: Personal Introduction

- Write a program that prints a short self-introduction:

- Expected output (use your own info):

```
"Hello! My name is [name]."
"I am [age] years old."
"I study Embedded Systems Engineering."
"I want to become an embedded software engineer!"
```

- Bonus: add a blank line between sections

Practice 7: Escape Sequences

- Print the following output using printf():

```
Name      Score

Kim       95



"End of list"
```

- Requirements:
- Use ₩t for tab alignment between columns
- Use ₩" for literal double-quote characters
- Use 4 separate printf() statements

Practice 8: Using puts()

- Rewrite the following code using puts() instead of printf():

```
printf("C Programming\n");
printf("Week 1 Practice\n");
printf("Hello, World!\n");
```

Hint: puts() adds ₩n automatically

Practice 9: char Data Type

- Declare: char grade = 'A';

- Print the following output:

```
Grade (character): A

ASCII value:       65
```

- Hint:
- %c prints the variable as a character
- %d prints the variable as an integer (ASCII code)

Practice 10: All Four Basic Types

- Declare variables of all 4 basic data types:

```
int    score   = 98
float  gpa     = 3.75f
double temp    = 36.5837
char   initial = 'K'
```

- Print each with the correct format specifier:

```
Score:    98
GPA:      3.75      (use %.2f)
Temp:     36.58     (use %.2f)
Initial:  K
```

Practice 11: Fix the Bug #2

- This code has 3 bugs. Find and fix them all:

```
#include <stdio.h>

int main(void)
{
    int    score   = 95

    printf("Score %f\n", score);
    Printf(”Passed!\n", score);
    return 0;
}
```

Practice 12: Diamond Pattern

- Print this diamond pattern using exactly 5 printf() statements:

```
  *
 ***
*****
 ***
  *
```

- One printf() per line
- Pay careful attention to spaces before *

Common Beginner Mistakes

- ❌ Missing semicolons at end of statements
- ❌ Forgetting #include <stdio.h>
- ❌ Using single quotes for strings ('Hello')
- ❌ Using Printf instead of printf
- ❌ Uninitialized variables (random garbage value)
- ❌ Ignoring compiler error messages

✅ Fix one error at a time — rebuild after each fix

How to Succeed in C Programming

- Type the code yourself — never just copy-paste
- Practice every day, even 30 minutes is enough
- Make mistakes intentionally and fix them
- Read error messages before asking for help
- Use the IDE debugger, not just guessing
- Help classmates — explaining reinforces learning

Week 1 Summary

- C: created 1972, still essential for systems programming
- IDE: Visual Studio (Win) or VS Code + extensions (Mac)
- Compilation: source code → compiler → executable
- Every C program: #include → main() → statements → return 0
- printf() prints to screen; ₩n starts a new line
- Errors: syntax (compile) · runtime · logic

What You Should Know Now

✓ What C is and why it matters for embedded systems

✓ How to install and use your IDE

✓ The structure of a C program

✓ How to use printf() and format specifiers

✓ The three types of errors

✓ How to read IDE error messages

Online Judge

임베디드시스템공학과 온라인 저지 게시판 문제 모음 채점 상황 열람 순위 대회 로그인

등록 정보

사용자 ID:	HJO202301676	이니셜 + 학번
닉네임:	황주옥	성명
비밀번호:	••••••••••••	비밀번호
비밀번호 반복 입력:	••••••••••••	
소속:		
이메일:	ghkdwndhr227@inu.ac.kr	이메일

Submit

Reset

임베디드시스템공학과 온라인 저지 | 게시판 | 문제 모음 | 채점 상황 열람 | 순위 | 대회 | HJO202301676

1 2 3 4 5 6 7 8 9 10

Problem ID | Go | Keywords Title or Source | Search

문제 ID	문제명	출처	AC	제출
1000	첫번째 문제		715	1104
1001	과일옮기기		200	283
1002	Hello World 두 줄에 출력해 보기		439	677
1003	따옴표 안에 Hello 출력하기		374	457
1004	큰 따옴표 안에 Hello World 출력하기		347	690
1005	채점서버 테스트 문제		223	300
1006	[문법-출력] 출력하기01	정보컴퓨터교사	426	490
1007	[문법-출력] 출력하기02	정보컴퓨터교사	395	449
1008	[문법-출력] 출력하기03	정보컴퓨터교사	405	538
1009	[문법-출력] 출력하기04	정보컴퓨터교사	383	467
1010	[문법-출력] 출력하기05	정보컴퓨터교사	368	520
1011	[문법-출력] 출력하기06	정보컴퓨터교사	373	705
1012	[문법-출력] 출력하기08	정보컴퓨터교사	92	521
1013	[문법-출력] 출력하기07	정보컴퓨터교사	336	515
1014	[문법-입출력] 정수 한 개 입력받아 그대로 출력하기	정보컴퓨터교사	268	327
1015	[문법-입출력] 문자 한 개 입력받아 그대로 출력하기	정보컴퓨터교사	226	265
1016	[문법-출력] 출력하기09	정보컴퓨터교사	107	445
1017	[문법-입출력] 실수 한 개 입력받아 그대로 출력하기	정보컴퓨터교사	200	607

임베디드시스템공학과 온라인 저지 게시판 문제 모음 채점 상황 열람 순위 대회 HJO202301676

1002: Hello World 두 줄에 출력해 보기

시간 제한: 1 Sec 메모리 제한: 128 MB
제출: 677 해결 문제 수: 439
[제출][채점 상황 열람][게시판]

문제 설명

이번에는 줄을 바꿔 문장을 출력하는 출력을 연습해보자.
다음과 같이 줄을 바꿔 출력 해야한다.

Hello
World

두 줄에 걸쳐서 줄을 바꿔 출력

입력

출력

Hello
World

출력 예시

```
Hello
World
```

도움말

출처

[제출][채점 상황 열람]

임베디드시스템공학과 온라인 저지 게시판 문제 모음 채점 상황 열람 순위 대회 HJO202301676

Problem **1002**

Language: C

```
#include <stdio.h>

int main() {

printf("Hello₩nWorld");

return 0;
}
```

Position: Ln 8, Ch 2 Total: Ln 8, Ch 64

Togg

출력:

```
Hello
World
```

입력:

출력:

```
SHOULD BE:
Hello
World
```

제출 TEST 状态 重置

임베디드시스템공학과 온라인 저지 게시판 문제 모음 채점 상황 열람 순위 대회 HJO202301676

Problem **1002**

Language: C

12 pt CPP

```c
#include <stdio.h>

int main() {
printf("Hello");

return 0;
}
```

Position: Ln 7, Ch 2 Total: Ln 7, Ch 56

☑ Toggle editor

입력:

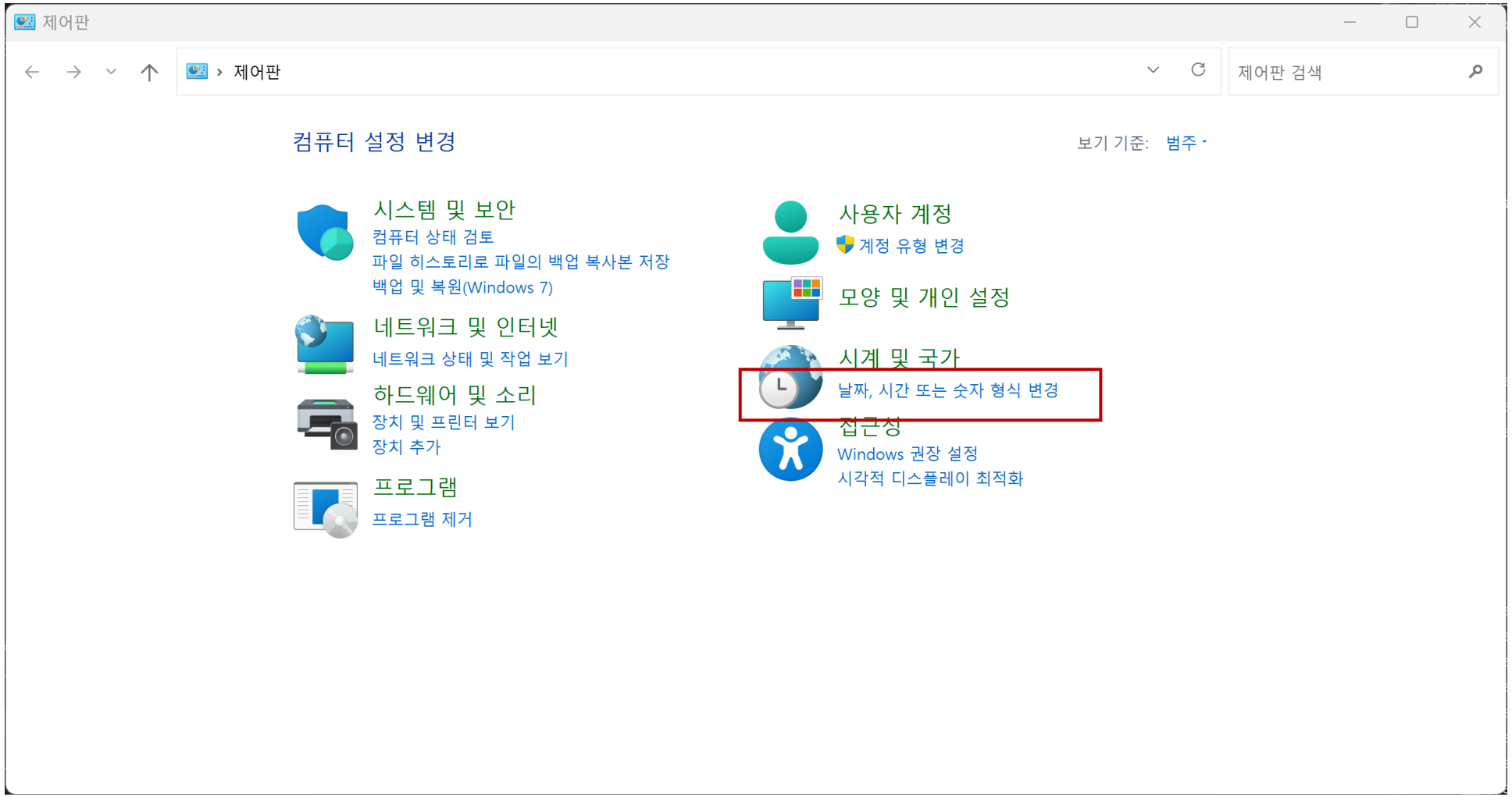
출력:

```
SHOULD BE:
Hello
World
```

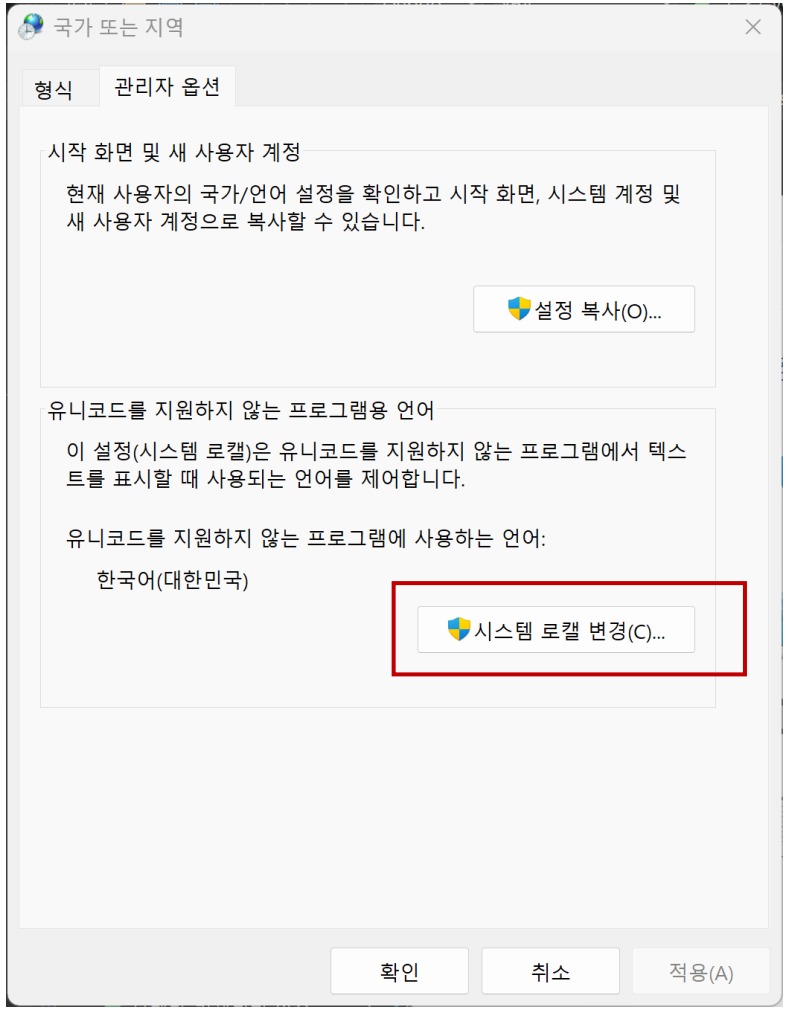
제출 TEST 状态 重置

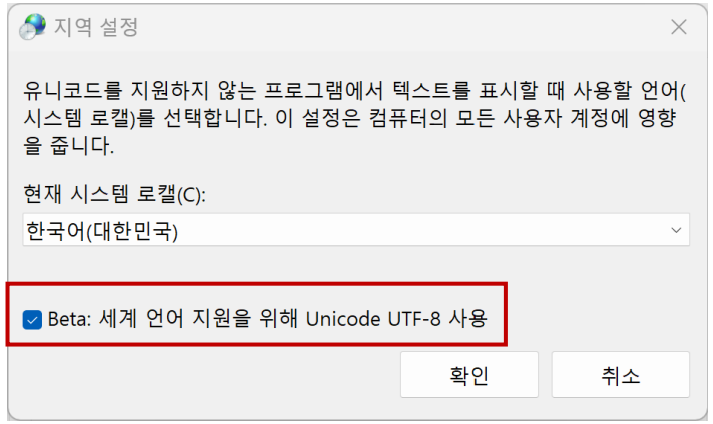
한글 출력 시 깨짐 현상 해결

Window key→ 제어판 열기



국가 또는 지역 선택 > 관리자 옵션 > 시스템 로캘 변경 > Beta: 세계 언어 지원 위해 Unicode UTF-8 사용





Human-Centered Neuroengineering LAB

C Programming (1)

2nd week. Variables and Data Types

Department of Embedded System Engineering
Incheon National University

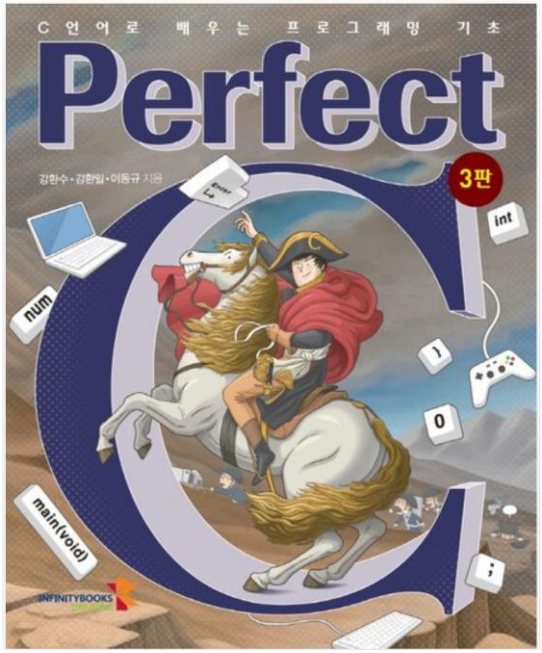
Instructor: Kyungho Won
kyungho.won@inu.ac.kr
Human-Centered Neuroengineering (HCN) Lab

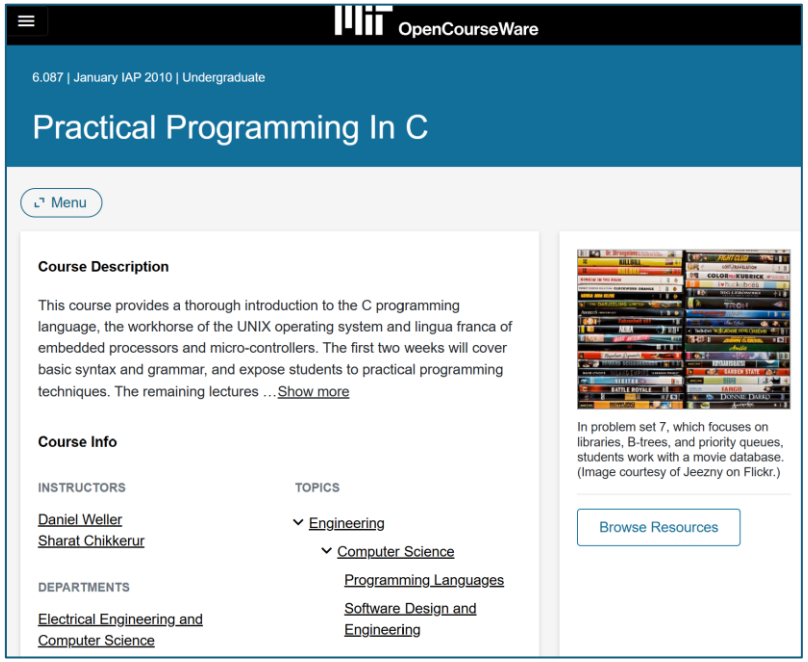
INU 인천대학교 INCHEON NATIONAL UNIVERSITY

* These lecture slides are for class use only. Unauthorized distribution or modification is prohibited.

Class Materials

- Lecture slides
  - Slides (PDF) will be distributed 1-hour earlier
- Perfect C 3판
  - 강환수, 강환일 이동규 저
  - 인피니티북스
- Practical Programming in C
  - Daniel Weller, Sharat Chikkerur
  - MIT OpenCouseWare





Course Syllabus (tentative)

1st week	- Introduction, Hello World, Compiling & Debugging	
2nd week	- Variables and Data Types	
3rd week	- Operators, Expressions, Preprocessor & I/O	
4th week	- Control Flow – Conditions (if, switch)	
5th week	- Control Flow – Loops (for, while, do-while)	
6th week	- Arrays	
7th week	- Functions	
8th week	Midterm	

Today's Agenda

- Review: Week 1 key concepts
- What are data types?
- Integer types: int, short, long, long long
- Signed and unsigned integers
- Floating-point types: float, double
- Character type: char and ASCII
- Variables: declaration, initialization, naming
- Constants: const and #define
- Type conversion and casting
- Getting user input: scanf() basics

Review: Week 1

- C program structure: #include → main() → return 0
- Compilation: source (.c) → compiler → executable (.exe)
- printf() prints to screen; ₩n = newline
- Three error types: syntax · runtime · logic
- IDE: Visual Studio (Win) / VS Code (Mac)

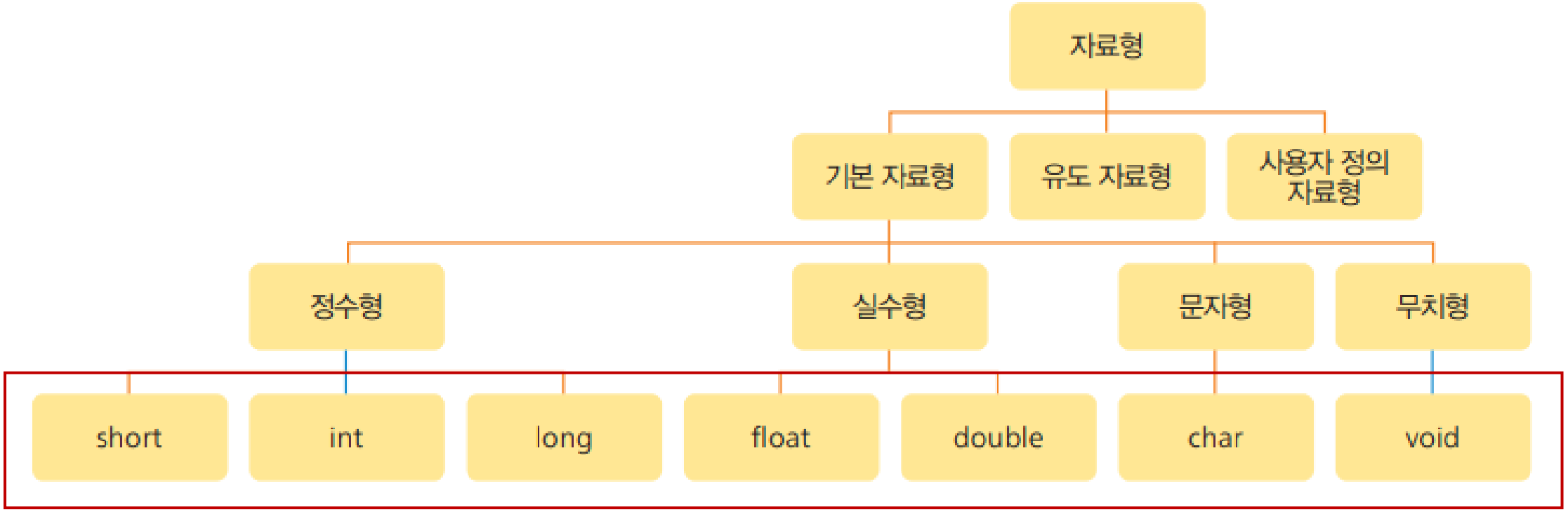
Data Types

What is a Data Type?

- Data type = tells the compiler:
- What kind of value is stored
- How much memory to reserve
- What operations are valid

- C is statically typed:
- → every variable must have a declared type
- → type cannot change after declaration

C Data Type Hierarchy



Basic types: integer · floating-point · character · void

Overview of Basic Types

- Integer: int, short, long, long long
- Unsigned: unsigned int, unsigned char, ...
- Floating-pt: float, double, long double
- Character: char
- Void: void (no value — used in functions)

- Each type has a different size and range
- → choosing the right type matters in embedded systems!

Memory and Types

- Every variable occupies memory (bytes):

- char → 1 byte = 8 bits
- short → 2 bytes = 16 bits
- int → 4 bytes = 32 bits (typically)
- long → 4 or 8 bytes
- double → 8 bytes = 64 bits

- Embedded systems have limited RAM — type size matters!

Integer Types

The int Type

- Most common integer type in C
- Typically 4 bytes (32 bits) on modern systems
- Range: −2,147,483,648 to +2,147,483,647

- int age = 20;
- int score = -5;
- int count = 0;

- ⚠ Uninitialized int holds garbage — always initialize!

int: Sizes and Limits

limits.h defines INT_MIN, INT_MAX, SHRT_MIN, LONG_MAX, etc.

```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    printf("int size:  %d bytes\n", (int)sizeof(int));
    printf("int min:   %d\n", INT_MIN);
    printf("int max:   %d\n", INT_MAX);
    return 0;
}
```

short, long, long long

- short — 2 bytes, range ±32,767
- short s = 100;

- long — at least 4 bytes (8 on 64-bit OS)
- long l = 1000000L; /* suffix L */

- long long — at least 8 bytes
- long long big = 9000000000LL; /* suffix LL */

- Use the smallest type that fits your data

Signed vs Unsigned

- signed → negative AND positive values (default)
- unsigned → zero and positive only

- signed int: −2,147,483,648 to +2,147,483,647
- unsigned int: 0 to +4,294,967,295

- Use unsigned for:
- Counts and sizes (can't be negative)
- Bit masks in embedded systems
- Hardware register values

Signed vs Unsigned Example

```c
#include <stdio.h>

int main(void)
{
    unsigned int u = 4294967295U;   /* max unsigned */
    int          s = -1;

    printf("unsigned: %u\n", u);    /* 4294967295 */
    printf("signed:   %d\n", s);    /* -1          */

    /* Mixing causes subtle bugs! */
    if (s < u)          /* signed -1 > unsigned max! */
        printf("s < u\n");
    else
        printf("s >= u  (unexpected!)\n");
    return 0;
}
```

Integer Overflow

- Overflow: value exceeds the type's range

→ wraps around silently (no crash, no warning!)

```
int x = 2147483647;    /* INT_MAX */
x = x + 1;             /* becomes -2147483648! */
```

- Critical in embedded systems:
- Sensor counter wraps: 255 + 1 = 0 for uint8_t
- Timestamps can overflow

→ always choose a type with enough range

Integer Literal Formats

- C supports three integer literal formats:

```
Decimal:     int a = 255;     /* normal */
Octal:       int b = 0377;    /* prefix 0 */
Hexadecimal: int c = 0xFF;    /* prefix 0x */
```

- All three equal 255 in decimal

- Hex is widely used in embedded systems:

→ hardware register addresses: 0x40021000

→ bit masks: 0xFF, 0x0F, 0b00001111

Integer Literals: Decimal, Octal, Hex

%X prints in uppercase hex, %x in lowercase

```
#include <stdio.h>

int main(void)
{
    int dec = 255;
    int oct = 0377;    /* octal   (prefix 0)  */
    int hex = 0xFF;    /* hex     (prefix 0x) */

    /* All three are the same value */
    printf("dec = %d\n", dec);  /* 255 */
    printf("oct = %d\n", oct);  /* 255 */
    printf("hex = %d\n", hex);  /* 255 */

    /* Print IN hex or octal */
    printf("in hex: %X\n", dec);  /* FF  */
    printf("in oct: %o\n", dec);  /* 377 */
    return 0;
}
```

Floating-Point Types

float and double

- float — 4 bytes, ~6-7 significant decimal digits
- float pi = 3.14f; /* suffix f */

- double — 8 bytes, ~15-16 significant decimal digits
- double pi = 3.14159265; /* default for literals */

- long double — 10 or 16 bytes (platform-dependent)

- Recommendation: use double unless memory is critical

float vs double Precision

float loses precision after ~7 significant digits

```c
#include <stdio.h>

int main(void)
{
    float  f = 3.1415926535897 93f;
    double d = 3.141592653589793;

    printf("float:  %.15f\n", f);
    /* 3.141592741012573  (inaccurate after 7 digits) */

    printf("double: %.15f\n", d);
    /* 3.141592653589793  (accurate to 15 digits) */
    return 0;
}
```

Scientific Notation Literals

Scientific notation: 1.5e6 = 1.5×10^6 = 1500000

```
#include <stdio.h>

int main(void)
{
    double big   = 1.5e6;    /* 1,500,000       */
    double small = 2.5e-3;   /* 0.0025          */
    double c     = 3.0e8;    /* speed of light  */

    printf("big:   %.0f\n",  big);
    printf("small: %.4f\n",  small);
    printf("c:     %.2e m/s\n", c);  /* 3.00e+08 */
    return 0;
}
```

Practice 1: Data Type Sizes

- Write a program that prints the size (bytes) of:
- short, int, long, long long, float, double, char

- Use sizeof() for each type
- Format: "int: 4 bytes" (use field width for alignment)

Practice 2: Temperature Converter

- Read a Celsius temperature from the user
- Convert to Fahrenheit: $F = (C \times 9 / 5) + 32$
- Print: "25.0C = 77.0F"

⚠ Watch out for integer division in C * 9 / 5!

- Use 9.0 and 5.0 to force floating-point division

Floating-Point Comparison Trap

- Floating-point numbers are approximations:
- 0.1 + 0.2 == 0.3 → FALSE in C!
- (actual stored value: 0.30000000000000004)

- Never use == to compare floats:

❌ if (x == 0.1) /* unreliable */

✅ if (fabs(x - 0.1) < 1e-9) /* correct */

- #include <math.h> for fabs()

Character Type

The char Type

- char = 1 byte, stores a single character
- char letter = 'A';
- char digit = '5';
- char space = ' ';
- char newline = '₩n';

- Single quotes for characters: 'A' ← char
- Double quotes for strings: "Hello" ← string (Week 7)

- Internally stored as its ASCII integer code

char and ASCII Values

'A'=65, 'a'=97 → difference is 32 (case conversion trick)

```c
#include <stdio.h>

int main(void)
{
    char c = 'A';

    printf("char:      %c\n", c);        /* A  */
    printf("ASCII:     %d\n", c);        /* 65 */
    printf("Next:      %c\n", c + 1);    /* B  */
    printf("Lowercase: %c\n", c + 32);   /* a  */

    /* Key ASCII values to remember: */
    /* 'A'=65  'a'=97  '0'=48  ' '=32 */
    return 0;
}
```

Useful ASCII Ranges

- Digits: '0' to '9' → ASCII 48 to 57
- Uppercase: 'A' to 'Z' → ASCII 65 to 90
- Lowercase: 'a' to 'z' → ASCII 97 to 122
- Space: ' ' → ASCII 32

- Useful checks:
- if (c >= '0' && c <= '9') → c is a digit
- if (c >= 'A' && c <= 'Z') → c is uppercase
- lowercase = uppercase + 32; → convert case

char Arithmetic Example

We'll learn for loops in later weeks — notice how char arithmetic works

```c
#include <stdio.h>

int main(void)
{
    char c;

    /* Print A to Z */
    for (c = 'A'; c <= 'Z'; c++)
        printf("%c ", c);
    printf("\n");

    /* Check if digit */
    char ch = '7';
    if (ch >= '0' && ch <= '9')
        printf("%c is a digit, value = %d\n", ch, ch - '0');

    return 0;
}
```

Practice 3: Character Info

- Read a single character from the user
- Print:
- The character itself
- Its ASCII code
- Whether it is a digit, uppercase, or lowercase

- Hint: use ranges like `c >= 'A' && c <= 'Z'`

Variables

Variable Declaration

변수선언은 컴파일러에게 이 소스에서 사용할 변수의 이름과 분류인 자료형을 알려주며, 컴파일러는 실제 변수선언 문장에 맞는 저장 영역을 메모리에 확보한다.

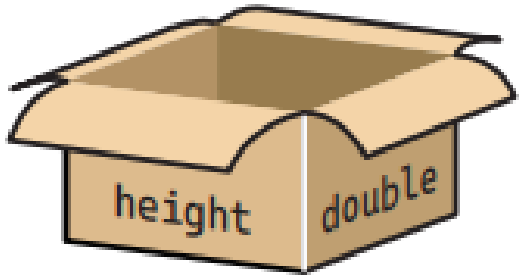
한 학과의 학생을 학번으로 구별하듯이 변수도 고유한 변수 이름으로 변수를 서로 구별할 수 있어야 한다. 일반적으로 한 함수에서 변수의 이름은 반드시 서로 구별되어야 한다.

변수선언문

자료형 변수이름

```
double height;
```

자료형 double, 변수이름 height로 실수를 저장할 수 있는 공간을 확보



type name; → reserves a named memory box of the correct size

Variable Declaration Syntax

- Syntax: type name;
- type name = initial_value;

```
int    age    = 20;
double salary = 3500.50;
char   grade  = 'B';
float  temp   = 36.5f;
```

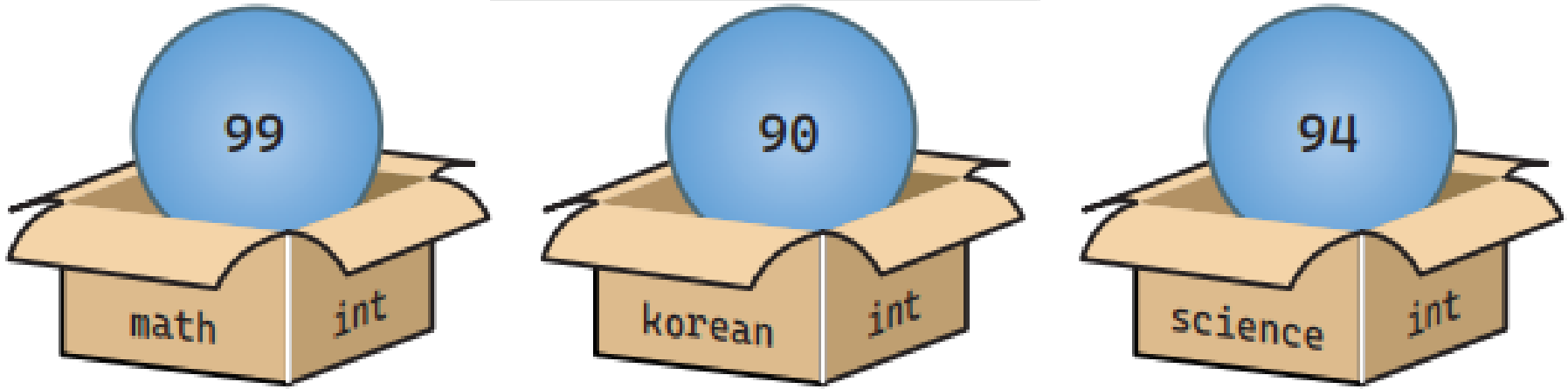
- Always initialize — uninitialized variables hold garbage!

int x; printf("%d", x); ← unpredictable output

Multiple Variables in Memory

```
int math = 99, korean = 90, science = 94;
```

```
int math = 99;
int korean = 90;
int science = 94;
```



Each variable gets its own named memory location

Declaring Multiple Variables

```
/* Separate lines — recommended for readability */
int math    = 99;
int korean  = 90;
int science = 94;

/* Same line — compact */
int math = 99, korean = 90, science = 94;

/* Declare first, assign later */
int a, b, c;
a = 10;
b = 20;
c = a + b;
```

Variable Naming Rules

✅ Valid names:

- Letters (a-z, A-Z), digits (0-9), underscore _
- Must start with a letter or _
- Case-sensitive: count ≠ Count ≠ COUNT

❌ Invalid names:

- 2fast (starts with digit)
- my-var (hyphen not allowed)
- int (reserved keyword)
- my var (no spaces)

Naming Best Practices

- Use descriptive names:

❌ int x, y, z;

✅ int width, height, area;

- Multi-word names: use snake_case

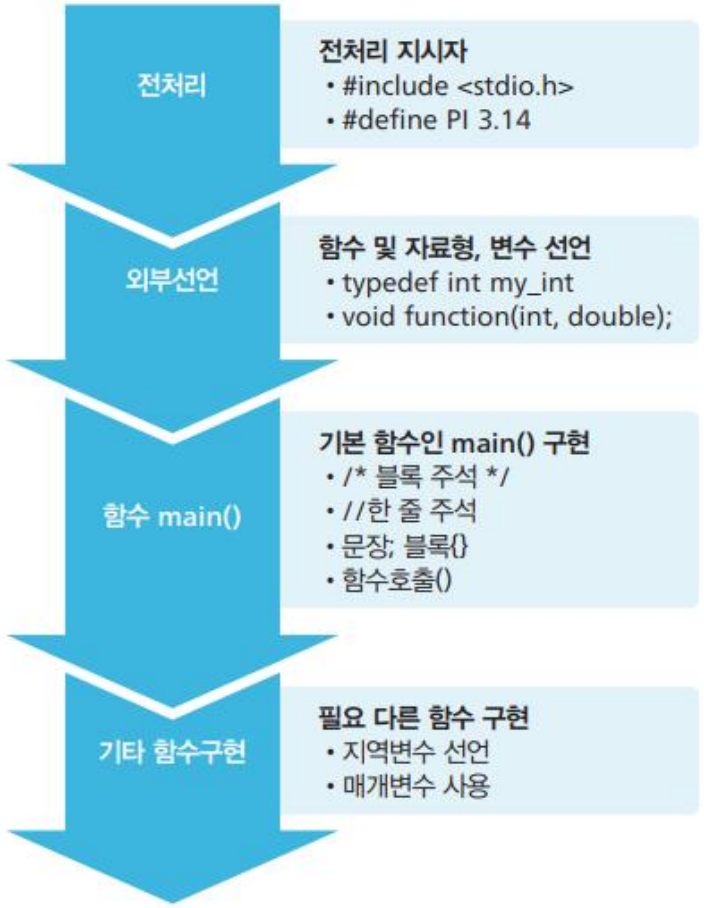
✅ student_count, max_speed, led_status

- Constants: ALL_CAPS

✅ MAX_SIZE, BAUD_RATE, PI

- Short names ok only for loop counters: i, j, k

Where Variables Live in a Program



Global (before main) vs Local (inside functions) — we'll cover scope in detail in later weeks

Local vs Global Variables (Preview)

Local variable: declared inside a function

- Only exists while that function runs
- Cannot be accessed from other functions

Global variable: declared outside all functions

- Exists for the entire program lifetime
- Accessible from any function

Best practice: prefer local variables

- → global variables cause hard-to-find bugs
- (Covered in depth in Week 6: Functions)